*Proof.* Under NTK regime, the model can be linearized around initialization:

$$\begin{aligned} F(\theta, x) &\approx F(\theta(0), x) + \langle \nabla_\theta F(\theta(0), x), \theta - \theta(0) \rangle \\ &= \underbrace{F(\theta(0), x)}_{\text{Initial Output}} + \Phi(x)^\top \Delta\theta \end{aligned}$$

where $\Phi(x) = \nabla_\theta F(\theta(0), x)$ and $\Delta\theta = \theta - \theta(0)$. The quadratic error term $\|\Delta\theta\|^2$ is $O(1/\text{width})$ and thus negligible.

Define centered labels $\tilde{y}_i = y_i - F(\theta(0), x_i)$. The problem reduces to ridge regression:

$$\min_{\Delta\theta} \frac{1}{2} \sum_{i=1}^{n} \left( \Phi(x_i)^\top \Delta\theta - \tilde{y}_i \right)^2 + \frac{\lambda}{2} \|\Delta\theta\|^2$$

with closed-form solution:

$$\Delta\theta^* = (\Phi\Phi^\top + \lambda I)^{-1} \Phi \tilde{\boldsymbol{y}} = \Phi^\top (\boldsymbol{H}^* + \lambda I)^{-1} \tilde{\boldsymbol{y}}$$

where $\boldsymbol{H}^* = \Phi\Phi^\top$ is the kernel matrix with $\boldsymbol{H}^*_{i,j} = \Theta_{\text{route}}(x_i, x_j)$.

The excess risk decomposes as:

$$\mathbb{E}[(F(\theta^*, x) - f^*(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f^*(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Variance}} + \sigma^2$$

- **Bias Term**:

  $$\text{Bias}^2 \leq \frac{\lambda^2}{n^2} \boldsymbol{y}^\top (\boldsymbol{H}^* + \lambda I)^{-2} \boldsymbol{y} \leq \frac{C_1 \lambda^2}{n} \boldsymbol{y}^\top (\boldsymbol{H}^* + \lambda I)^{-2} \boldsymbol{y}$$

  where $C_1$ absorbs constants related to the magnitudes of $g_k(x)$ and expert gradients.

- **Variance Term**: The parameter covariance is $\text{Cov}(\Delta\theta^*) = \sigma^2 (\Phi\Phi^\top + \lambda I)^{-1} \Phi\Phi^\top (\Phi\Phi^\top + \lambda I)^{-1}$. For prediction variance:

  $$\text{Variance} = \mathbb{E}_x[\Phi(x)^\top \text{Cov}(\Delta\theta^*) \Phi(x)] = \frac{\sigma^2}{n} \text{Tr}(\boldsymbol{H}^* (\boldsymbol{H}^* + \lambda I)^{-1})$$

- **Confidence Term**: By standard concentration inequalities (e.g., for sub-Gaussian noise),

  $$\mathbb{P}\left( \left| \frac{1}{n} \sum_{i=1}^{n} \epsilon_i^2 - \sigma^2 \right| \geq t \right) \leq e^{-nt^2/(2\sigma^4)} \implies \frac{\sigma^2 \log(1/\delta)}{n}$$

Collecting all components and adjusting constants $C_1, C_2$ completes the proof. $\square$

### C.4 Connecting Edit Distance to Effective Dimension

The bound shows that simple (low-dimensional) routing kernels generalize better. We now empirically test how routing pattern complexity (via edit distance) correlates with this effective dimension $d_{\text{eff}} = \text{Tr}(\boldsymbol{H}^* (\boldsymbol{H}^* + \lambda I)^{-1})$.

Our setup involves a lightweight MoE architecture with 16 experts and a 100-dimensional input space. Input data are synthetic Gaussian clusters, providing a controlled environment to analyze pre-defined routing strategies. For each experimental trial, 200 samples are split into training and validation sets. We train a one-layer MoE model with ReLU activations where only expert networks are trainable; the router's weights are initialized once and then frozen. To explore diverse fixed routing patterns, we perform multiple independent runs, *each starting with a different random initialization of the router*.

Following training, we extract the average edit distance of the pre-determined expert routing paths and the effective dimension. As illustrated in Figure 8, we observe a strong positive correlation between these two quantities. This empirically demonstrates that the structural properties of fixed routing, measured by edit distance, meaningfully impact the complexity of the function space learned by the experts, as predicted by $d_{\text{eff}}$. This alignment supports edit distance as a valuable diagnostic for MoE generalization.

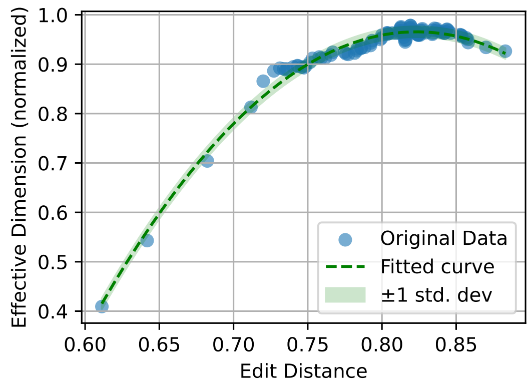


Figure 8: **Correlation of Edit Distance with Effective Dimension.** Each point represents an independent experimental run where the router is initialized with a different random seed and then fixed for expert training. This varying initialization leads to diverse fixed routing patterns. We observe a strong positive correlation. This suggests that the structural diversity of pre-defined routing significantly impacts the complexity of the function space learned by experts.

# D Routing Dynamics Analysis

To investigate whether routing evolution arises primarily from router-layer updates or other weight updates, we analyze convergence speed and stability across parameter types.

**Results.** Table 3 reports convergence speed (magnitude drop-off over time) and stability (consistency of updates). Router weights adapt fastest while expert weights stabilize most gradually, indicating a division of labor: routing decisions are steered by fast-updating routers atop slow-evolving expert features. Attention layers lie in between.

Table 3: Convergence speed and stability across parameter types. Higher values indicate faster convergence or greater stability.

| Parameter Type | Convergence Speed ↑ | Stability ↑ |
|---|---|---|
| Router | **0.151** (fastest) | 5.63 |
| Expert | 0.063 (slowest) | **7.32** |
| Attention | 0.083 | 6.01 |

**Layerwise Routing Dynamics.** To localize routing evolution, we compute the edit distance of routing assignments across layers. Layers 0–3 exhibit the most post-convergence change, suggesting that early layers remain active in refining pathways.

**Metric Computation.** We periodically snapshot parameter L2 norms during training. For each parameter type, we compute the mean absolute norm change between checkpoints, normalized by average scale. *Convergence speed* is defined as the drop in change magnitude from early to late training, while *stability* is the inverse standard deviation of these normalized changes. This simple but robust analysis consistently reveals router–expert asymmetries.

# E Analysis

## E.1 Effect of Routing-Weight Threshold on Pathway Dynamics

To examine the robustness of our findings with respect to the routing-weight threshold used for pathway construction, we repeat the analysis in Fig. 4 under different cumulative routing-weight thresholds ranging from 0.5 to 0.8 (interval 0.1). This threshold determines how many experts are considered *active* in each layer when forming the pathway sequence: a higher threshold includes more experts, while a lower threshold selects only the most dominant ones.

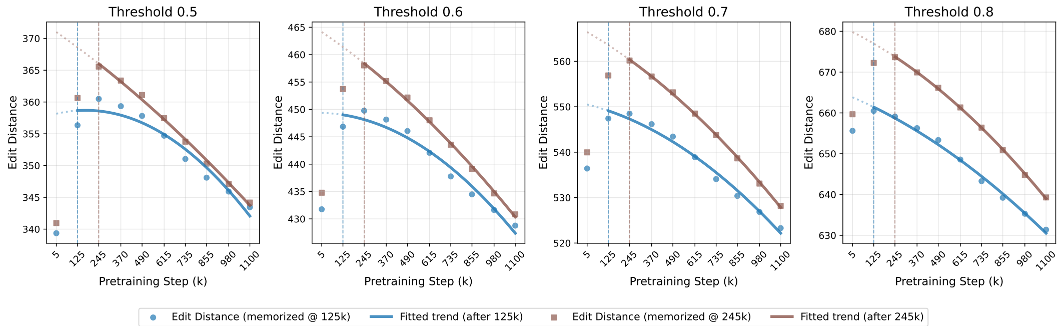


Figure 9: **Effect of routing-weight threshold on pathway edit distance on domain-specific QA tasks.** Each panel shows the average pathway edit distance for this domain under a different cumulative routing-weight threshold (0.5–0.8, interval 0.1). Higher thresholds include more active experts, leading to longer absolute distances, but all settings exhibit the same qualitative pattern: pathway edit distance decreases consistently after memorization, confirming that the observed dynamics are robust to the threshold choice.

Figure 9 presents the results for the domain-specific QA tasks. Although the absolute values of edit distance increase with larger thresholds—reflecting the longer sequences formed when more experts are included—the qualitative trends remain consistent across all settings. For every threshold, the pathway edit distance decreases after memorization (marked by vertical dashed lines), and the decrease becomes more pronounced at later checkpoints.

These results demonstrate that our conclusions are **robust to the choice of routing-weight threshold**. The observed reduction in pathway complexity after memorization, corresponding to the **emergence of shared pathways**, persists across all threshold settings.

### E.2 Effect of LoRA Rank on Pathway–Generalization Correlation

To examine whether the correlation between pathway metrics and generalization depends on the configuration of instruction tuning, we conduct an ablation on the LoRA rank used during lightweight finetuning. As described in Appendix B, the default setup adopts a rank of 32 for all domains. Here, we additionally train a LoRA adapter with rank 64 on the *commonsense* domain, following the same data, training steps, and hyperparameters as in the main experiments.

Table 4 reports the correlation between pathway complexity metrics and downstream test accuracy under the two LoRA ranks. The results show that increasing the rank from 32 to 64 yields nearly identical correlations for both *Pathway Distance* and *Pathway Consistency*, while training and validation losses remain weakly correlated. This consistency indicates that the observed relationships between pathway metrics and generalization are **robust to LoRA configuration** and primarily reflect intrinsic pretraining-phase dynamics rather than artifacts of instruction tuning.

### E.3 Effect of Memorization Thresholds $\varepsilon$ and $\delta$

We conduct an ablation study on the two hyperparameters that define the memorization criterion in Section 3.1: the loss threshold $\varepsilon$ and the stability threshold $\delta$. Specifically, for a sample $x_i$ with loss $\ell_i(\theta_t)$ at pretraining step $t$, it is considered *memorized* from step $t_i^*$ if $\ell_i(\theta_t) \leq \varepsilon$ and $|\ell_i(\theta_t) - \ell_i(\theta_T)| \leq \delta$ for all $t \geq t_i^*$. We evaluate how sensitive the pathway trends are to these two parameters using the domain-specific QA task.

**Varying $\varepsilon$.** Figure 10 shows results when fixing $\delta = 0.05$ and varying $\varepsilon$ from 1.5 to 2.0. Across all settings, the pathway edit distance after memorization consistently decreases with training, and the overall trend remains stable—indicating that $\varepsilon$ only shifts the point where samples are considered memorized but does not affect the post-memorization dynamics.